IAM Analyst Internship Program

Auth0 Identity, Access & API Security Track — Basic to Advanced

Portfolio Capstone: Enterprise Identity & Access Management Platform for a Healthcare SaaS Lab

A practical, self-directed internship-style program designed around the Auth0 environment you already have. The program emphasizes skills that can be demonstrated in IAM Analyst, Identity Engineer, Junior IAM, CIAM, and IT Security interviews.

Core stack: Auth0 Free Plan + Python + FastAPI/Flask + SQLite + Git + GitHub + Postman/Bruno + your existing Active Directory lab.
Free-first design: Every required module uses free/open-source tooling. Paid-only Auth0 features are not required for completion. Where the Auth0 Free Plan has limits, the lab uses standards-based alternatives or marks the feature as optional.

Program	Recommended pace	Difficulty
10 modules + capstone	6–8 weeks, 8–12 hrs/week	Basic → Advanced
Evidence	Scripts + screenshots + test results + documentation	Interview-ready

Important: This is a self-directed training curriculum modeled on common IAM workflows. It is not an official Okta/Auth0 certification or employer internship.

1. Program Overview

The supplied internship document is structured as a progressive IAM Analyst track: environment setup, user/group lifecycle, RBAC, security controls, SSO, MFA, automation/reporting, and an audit-oriented capstone. This program keeps that progression but adapts it to the Auth0 tenant shown in your screenshot and adds a local application/API layer so the project remains meaningful even when an Auth0 feature is not available on the Free plan.

> The original document explicitly requires module-by-module work, with scripts, screenshots, and short write-ups treated like change-ticket evidence. This program follows the same discipline. filecite turn0file0 L8-L17

Recommended outcome

- Build one coherent IAM system instead of several unrelated mini-projects.
- Demonstrate authentication, authorization, lifecycle management, SSO, MFA, API security, automation, logging, and access review.
- Keep a GitHub evidence trail so every resume bullet can be demonstrated during an interview.
- Use only synthetic users and non-sensitive data.

Target job roles

- IAM Analyst / Identity & Access Management Analyst
- Junior Identity Engineer / IAM Engineer
- Okta/Auth0 Administrator or Support Analyst
- Identity Security / IT Security Analyst
- Junior CIAM / Authentication Engineer

2. Free Lab Architecture & What Is Actually Free

Auth0 currently lists a Free plan at $0/month with no credit card required to sign up. Its current public pricing lists 25,000 monthly active users, 5 Organizations, 1 Enterprise Connection, inbound SCIM, passwordless authentication, custom attributes, 5 Actions, and community support. It also lists Role Management as not included on the Free plan. □cite□turn0search0□turn0search2□

> Therefore, this curriculum deliberately does not make Auth0 Core RBAC a required paid feature. Instead, authorization is demonstrated with API scopes and a small local authorization layer using claims/app metadata where appropriate. This keeps the project reproducible on the Free plan.

> Your screenshot shows a 22-day trial notice. Auth0 states that the trial runs for 22 days and the Free plan activates automatically afterward; trial-only features do not remain after the trial. □cite□turn0search8□

Required free tools

Tool	Use	Cost
Auth0 Free	Identity, Universal Login, OAuth/OIDC, APIs, Actions, users, connections	Free
Python 3.x	Automation and IAM scripts	Free/open source
FastAPI or Flask	Protected API for authorization tests	Free/open source
SQLite	Local audit/access-review data	Free
VS Code	Development	Free
Git + GitHub	Version control and portfolio	Free tiers
Postman or Bruno	API testing	Free options
draw.io	Architecture diagrams	Free web/desktop options
Windows Server AD lab	Optional hybrid identity extension	Use your existing lab

> Do not use real employee data, real customer records, production credentials, or secrets in this lab.

3. Capstone Scenario — SecureHealth IAM

You are the junior IAM analyst for a fictional healthcare SaaS company, SecureHealth Technologies. The company has employees, internal applications, a customer-facing API, and an audit requirement. Your job is to build an identity layer that controls who can sign in, what they can access, how access changes over the employee lifecycle, and how access is reviewed.

```
SecureHealth IAM Lab
|
+---------------------+----------------------+
| |
Users Applications
| |
v v
Auth0 <---- OAuth 2.0 / OIDC ----> Web App / API
| |
+-------+--------+ |
| | | v
MFA Actions Enterprise/SCIM* JWT validation
| | | |
+-------+--------+ v
| Permissions
v |
Identity Lifecycle v
JML + Access Review ALLOW / 403 DENY

* Enterprise/SCIM modules are included only if your Free tenant exposes the
required capability; otherwise complete the standards/documentation path.
```

Business requirements

- New users must receive the minimum access required for their job.
- A department/role change must remove obsolete access before granting new access.
- Leavers must lose access promptly and leave an audit trail.
- Applications must validate access tokens rather than trusting client-side claims.
- Privileged operations must require stronger authentication controls where supported.
- IAM analysts must be able to produce an access-review report and explain findings.

Suggested synthetic identities

User	Department	Job role	Expected access
alice.hr	HR	HR Analyst	Employee read/update
rahul.finance	Finance	Finance Analyst	Transaction read
priya.it	IT	IT Engineer	System read/update
john.sales	Sales	Sales Analyst	Sales report read
iam.admin	IT	IAM Administrator	Identity administration

4. Internship Progression

Module	Focus	Level	Main evidence
1	Environment + IAM design	Basic	Architecture + repo
2	Users + identity lifecycle	Basic	Users + lifecycle SOP
3	Authorization + least privilege	Basic → Intermediate	Scopes + claims + test matrix
4	OAuth 2.0 + OIDC + JWT	Intermediate	Working web/API flow
5	MFA + authentication security	Intermediate	Policy/test evidence
6	SSO + Enterprise Connection/SCIM	Intermediate → Advanced	SSO/SCIM evidence or documented alternative
7	JML automation	Advanced	Python automation + logs
8	Audit, logs + access review	Advanced	Access review + findings
9	AD + Auth0 hybrid extension	Advanced	Architecture + sync design
10	Capstone + interview package	Capstone	Final portfolio + demo

The supplied source progresses from AD/OU setup through user/group lifecycle, RBAC, GPO, SSO, MFA, automation/reporting, and a final access review. This program intentionally mirrors that increasing difficulty while moving the implementation center to Auth0. □filecite□turn0file0□L19-L40□ □filecite□turn0file0□L67-L92□ □filecite□turn0file0□L120-L164□ □filecite□turn0file0□L165-L218□

Module 1 — Lab Setup & IAM Architecture [BASIC]

Objective: Create a clean, reproducible lab and document how identities move through Auth0 to protected applications.

Free tools

- Auth0 Free tenant
- VS Code
- Python 3.x
- Git/GitHub
- draw.io
- Postman/Bruno

Tasks

1. Record your Auth0 tenant domain without putting secrets in GitHub.
2. Create a Git repository named securehealth-auth0-iam.
3. Create folders: docs/, diagrams/, scripts/, api/, tests/, screenshots/, reports/.
4. Draw the capstone architecture showing user → Auth0 → token → API → authorization decision.
5. Create a lab inventory listing tenant domain, local tools, Python version, and test application names. Never record client secrets.
6. Create a README with objectives, scope, assumptions, and a warning that all identities/data are synthetic.

Suggested repository

```
securehealth-auth0-iam/
■■■ README.md
■■■ docs/
■■■ diagrams/
■■■ scripts/
■■■ api/
■■■ tests/
■■■ reports/
■■■ screenshots/
```

Deliverable

Architecture diagram + README + lab inventory + initial Git commit.

Interview questions

- What problem does an IdP solve?
- What is the difference between authentication and authorization?
- Where should secrets be stored?
- Why should IAM changes be documented?

Module 2 — User & Identity Lifecycle Management [BASIC → INTERMEDIATE]

Objective: Practice the core IAM analyst lifecycle: create, update, suspend/deactivate, and audit identities.

Tasks

1. Create the five synthetic users listed in the scenario. Use a consistent naming convention.
2. Add useful non-sensitive profile attributes such as department and job title where your tenant permits them.
3. Create a lifecycle worksheet with Joiner, Mover, and Leaver actions.
4. Test a user profile update and record the expected downstream access change.
5. Deactivate one test user and verify the user can no longer authenticate.
6. Document the difference between deleting a user and deactivating a user.
7. Repeat the same lifecycle using the Auth0 Management API where your tenant exposes the required endpoint and credentials.

Suggested lifecycle matrix

Event	Identity action	Access action	Evidence
Joiner	Create user	Assign baseline access	User + access test
Mover	Update department/role	Remove old access; grant new	Before/after
Leaver	Block/deactivate	Revoke application access	403/login failure + log

Deliverable

Lifecycle SOP + screenshots + test results + one Python script for a safe user lookup/update operation.

Source alignment

The supplied curriculum treats new-hire provisioning and offboarding as two of the most common IAM analyst tasks and requires scripts, sample data, and logs. filecite turn0file0 L40-L66

Module 3 — Least Privilege & Authorization Design [INTERMEDIATE]

Objective: Build a formal authorization model without relying on paid Auth0 Core RBAC features.

Free-plan strategy

Auth0's current Free plan lists Role Management as not included. cite turn0search0 Therefore, the required lab uses API scopes + application metadata/custom claims + authorization checks in the API. If your tenant currently exposes role management during the trial, you may demonstrate it as an optional comparison, but do not make it a required dependency.

Design

```
Department Permission examples
HR read:employees, update:employees
Finance read:transactions
IT read:systems, update:systems
Sales read:reports
IAM Admin read:users, manage:access
```

Tasks

1. Create a permission matrix: user type → API scope → endpoint → expected decision.
2. Register a custom API in Auth0 and define the minimum scopes needed for the capstone.
3. Configure your API to validate JWT signature, issuer, audience, expiry, and required scope.
4. Implement a simple authorization function in Python that returns 403 when a caller lacks the required permission.
5. Create positive and negative tests: HR accessing HR data should succeed; Finance accessing HR data should be denied.
6. Document least privilege and separation of duties.

Deliverable

Role/permission matrix + API scope configuration + Python authorization module + test report with at least three ALLOW and three DENY cases.

Interview questions

- RBAC vs ABAC?
- Authentication vs authorization?
- 401 vs 403?
- Why is least privilege important?
- Why must the API validate the token itself?

Module 4 — OAuth 2.0, OIDC & JWT Security [INTERMEDIATE]

Objective: Understand the protocols behind modern application authentication and API authorization.

Tasks

1. Create a simple local web application using FastAPI/Flask.
2. Configure it as an Auth0 application using the flow appropriate to the app type.
3. Implement Universal Login and OIDC sign-in.
4. Capture and explain the ID token and access token. Never paste a real token into GitHub.
5. Decode a sample JWT locally and identify iss, aud, exp, sub, scope and other relevant claims.
6. Call the protected API using a valid access token.
7. Repeat the request with no token, an expired token, a wrong audience, and insufficient scope; document the result.

```
User
↓
Web App
↓
Auth0 /authorize
↓
Authentication
↓
Authorization Code + PKCE
↓
Token exchange
↓
ID Token + Access Token
↓
Protected API
↓
JWT + scope validation
↓
ALLOW / 403
```

Deliverable

OIDC flow diagram + working local app + API test collection + token-claim explanation + troubleshooting notes.

Interview questions

- OAuth 2.0 vs OIDC?
- ID token vs access token?
- What is PKCE?
- What are issuer and audience?
- Why shouldn't an API accept an ID token as an API access token?

Module 5 — MFA & Authentication Security [INTERMEDIATE]

Objective: Add stronger authentication and document the security trade-offs.

Tasks

1. Review the authenticators/factors available in your Auth0 Free tenant.
2. Enable one free/available MFA method and enroll a synthetic test user if the tenant exposes the feature.
3. Create a test case for normal authentication and MFA-protected authentication.
4. Document enrollment, recovery, failed-factor behavior, and user support considerations.
5. Compare password-only, MFA, and passwordless approaches.
6. Record which advanced MFA features are trial/paid-only in your tenant so the portfolio is honest.

Suggested comparison

Method	Security	User friction	Lab status
Password only	Baseline	Low	Required
MFA factor available on tenant	Stronger	Medium	Required if exposed
Passwordless	Modern	Low/medium	Optional
Advanced/pro factors	Depends	Depends	Do not require for free capstone

Deliverable

Authentication security design + screenshots + test cases + feature availability note.

Source alignment

The supplied program emphasizes MFA as an enforceable security control and asks for documented policy design, testing, and sign-in evidence. □filecite□turn0file0□L143-L164□

Module 6 — Enterprise SSO & SCIM [INTERMEDIATE → ADVANCED]

Objective: Understand federation and lifecycle provisioning between an identity provider and SaaS applications.

Current free-plan note

Auth0's current Free plan lists one Enterprise Connection and inbound SCIM as included. cite turn0search0 turn0search6 Availability can depend on the exact tenant/product mode, so if a menu is not exposed, complete the documented architecture and local mock-integration path rather than purchasing a plan.

Tasks — SSO

1. Identify Auth0 as the IdP in the lab scenario and a local/test application as the service provider.
2. Configure the available enterprise/SAML connection only if your tenant exposes it.
3. Document IdP, SP, redirect/ACS, metadata, claims, and logout considerations.
4. Test sign-in and record the authentication path.

Tasks — SCIM

1. Read the SCIM user/group lifecycle model: create, read, update, deactivate.
2. If inbound SCIM is exposed in your tenant, connect it to a controlled test endpoint or supported integration.
3. If not exposed, implement a small local SCIM-style mock endpoint in FastAPI that accepts user lifecycle requests and logs them.
4. Document how SCIM differs from SAML: SAML authenticates; SCIM provisions identity data.

Deliverable

SSO flow diagram + SCIM lifecycle diagram + configuration screenshots or a working local SCIM mock + troubleshooting notes.

Module 7 — JML Automation with Python [ADVANCED]

Objective: Turn the manual lifecycle process into repeatable automation with validation and logging.

Automation design

```
Joiner CSV / request
↓
Validation
↓
Create / update identity
↓
Apply department access metadata
↓
Record change
↓
Verification report

Mover request → remove old access → add new access → verify
Leaver request → deactivate → revoke/expire access where supported → verify
```

Tasks

1. Create employees.csv with 10–15 synthetic employees.
2. Write provision_user.py with input validation and safe error handling.
3. Write move_user.py that requires both old and new department values and records before/after access.
4. Write offboard_user.py that deactivates the user and records a timestamp.
5. Use environment variables for secrets; never hard-code client secrets.
6. Add structured JSON or CSV logging.
7. Run at least five test cases including a duplicate user, invalid department, missing field, successful mover, and successful leaver.

Deliverable

Three Python scripts + sample CSV + log file + test evidence + short runbook.

Interview questions

- How do you prevent duplicate provisioning?
- What happens if step 3 fails after step 2 succeeds?
- How do you make an IAM script idempotent?
- What should be logged?
- How do you protect API credentials?

Module 8 — Audit Logs, Access Review & Findings [ADVANCED]

Objective: Simulate the recurring access-review work performed by IAM and security teams.

Tasks

1. Identify the logs/events available in your Auth0 tenant and local API.
2. Create an access-review dataset containing username, department, permissions, application access, MFA status (where available), and last relevant activity.
3. Create three intentional lab findings: stale user, excessive permission, and lifecycle mismatch.
4. Export the access-review data to CSV.
5. Write a Python report generator that produces an HTML or Markdown summary.
6. Remediate at least one finding and capture before/after evidence.
7. Write an executive summary: scope, findings, remediation, residual risk.

Finding template

Field	Example
Finding ID	IAM-001
Risk	Medium/High
Observation	Finance test user has HR permission
Impact	Unauthorized access to employee data
Recommendation	Remove HR permission; verify least privilege
Evidence	Before/after API test
Owner	IAM Analyst
Status	Remediated

Deliverable

Access-review CSV + report + three findings + one remediation + executive summary.

Source alignment

The supplied capstone asks for full access reports, intentional findings, live remediation, and an executive summary. □filecite□turn0file0□L191-L218□

Module 9 — Active Directory + Auth0 Hybrid Identity Extension [ADVANCED]

Objective: Connect your existing Windows Server Active Directory knowledge to cloud/CIAM concepts and learn where the responsibilities differ.

Architecture

```
On-prem AD
|
| identity source / federation concept
v
Identity integration layer
|
v
Auth0
|
+---- Web Application
|
+---- Protected API
|
+---- Audit / lifecycle automation
```

Tasks

1. Document your existing AD domain, OUs, department groups, and role groups without exposing personal data.
2. Map AD attributes to Auth0-style identity attributes.
3. Create an AD-to-cloud identity mapping table: username, department, title, status, manager.
4. Document Joiner-Mover-Leaver across AD and Auth0: source-of-truth, sequencing, failure handling, and audit evidence.
5. Where direct AD integration is unavailable on your free tenant, produce the architecture and simulate synchronization with CSV/Python.
6. Explain the difference between Workforce IAM (employees) and CIAM (customers/end users).

Deliverable

Hybrid architecture diagram + attribute mapping + JML sequence + failure-handling document.

Interview questions

- What is the source of truth?
- Why can deprovisioning order matter?
- How would you handle a failed sync?
- Workforce IAM vs CIAM?
- Why should identity status be reconciled regularly?

Module 10 — Capstone: Enterprise IAM Access Review & Security Demo [CAPSTONE]

Objective: Combine Modules 1–9 into one audit-ready portfolio project that a hiring manager can understand in 5–10 minutes.

Capstone scenario

A new SecureHealth employee joins Finance, later moves to IT, and eventually leaves. During the quarter, an access review discovers that a test account retained a Finance permission after moving to IT. You must detect, explain, remediate, and prove the correction.

Capstone workflow

```
1. Joiner
Create identity → baseline access → authenticate → API access

2. Mover
Finance → IT → remove Finance permission → add IT permission → test

3. Leaver
Deactivate identity → test login failure → record evidence

4. Audit
Pull users/access → identify 3 findings → remediate one

5. Security
Test missing/invalid token, wrong audience, insufficient scope, MFA case

6. Final review
Produce report + architecture + scripts + evidence + executive summary
```

Required final deliverables

- Architecture diagram
- README with setup and test instructions
- User lifecycle SOP
- Permission matrix
- OAuth/OIDC flow diagram
- MFA/security test evidence
- JML Python automation
- API authorization tests
- Access-review CSV/HTML report
- Three audit findings and one remediation
- Executive summary
- 5–8 minute demo script

Pass criteria

Area	Minimum evidence
Identity	Create/update/deactivate test user
Authentication	Working OIDC login
Authorization	3+ allowed and 3+ denied API tests
Security	Token validation + MFA/passwordless test where available
Lifecycle	Joiner + mover + leaver evidence

Area	Minimum evidence
Automation	Python scripts + logs + error handling
Audit	Access report + 3 findings + remediation
Portfolio	Clean GitHub repo + architecture + README

5. Day-by-Day Execution Plan

Day	Activity	Output
1	Auth0 tenant orientation; record domain; create Git repo	Repo + inventory
2	Architecture + business requirements	Diagram
3	Create synthetic users	Users + screenshots
4	Lifecycle worksheet + deactivate/reactivate test	Lifecycle SOP
5	Create API + scopes	API config
6	Build authorization checks	Python module
7	OIDC web app	Working login
8	JWT/API validation	API tests
9	MFA/passwordless review	Security evidence
10	SSO/SCIM exploration	SSO/SCIM docs
11	SCIM mock if required	FastAPI endpoint
12	JML automation	3 Python scripts
13	Audit data + findings	Access review
14	AD/Auth0 hybrid mapping	Hybrid diagram
15	Capstone scenario run	Before/after evidence
16	README + cleanup	Portfolio repo
17	Interview demo preparation	Demo script
18	Mock IAM interview	Answers + gap list

If you have 6–8 weeks, spend several days per module instead of following this compressed schedule.

6. First-Day Setup — Exact Click-by-Click Checklist

You already have an Auth0 tenant visible in the dashboard. Your screenshot shows the left navigation with Applications, Authentication, Organizations, User Management, and Users. That is enough to begin the project.

A. Protect the tenant

1. Do not enter billing information just to complete this lab.
2. Do not use real employee/customer information.
3. Do not commit client secrets, API tokens, passwords, or real access tokens to GitHub.
4. Create a separate browser profile if useful for keeping the lab isolated from personal accounts.

B. Create the GitHub repository

```
Repository name:
securehealth-auth0-iam

Initial folders:
docs
diagrams
scripts
api
tests
reports
screenshots
```

C. Create the first Auth0 API

1. In Auth0 Dashboard, open Applications → APIs.
2. Choose Create API.
3. Name it SecureHealth Employee API.
4. Use a lab-only audience/identifier such as https://securehealth.local/api.
5. Choose the signing algorithm offered by the tenant; keep the default recommended asymmetric option unless your lab requirements say otherwise.
6. Add only the scopes you need: read:employees, update:employees, read:transactions, read:systems, update:systems, read:reports.
7. Save and capture a screenshot with secrets hidden.

D. Create the first test user

1. Open User Management → Users.
2. Create rahul.finance using a test email address you control.
3. Set department = Finance and title = Finance Analyst if your profile supports those fields.
4. Do not reuse an important personal password.
5. Verify the user can sign in only through the intended test application.

E. Create the first API test

1. Obtain an access token through your configured application flow.
2. Call a protected endpoint with the token.
3. Record the status code and response.
4. Remove the scope or use the wrong scope and repeat.
5. Expected outcome: the API should reject unauthorized access with a clear 401/403 behavior.

This first-day lab creates the foundation for every later module.

7. Safe Starter Code Patterns

These are intentionally small patterns. You will expand them only after understanding the IAM flow.

Environment variables

```
# Windows PowerShell example
$env:AUTH0_DOMAIN = "your-tenant.us.auth0.com"
$env:AUTH0_CLIENT_ID = "lab-client-id"
$env:AUTH0_CLIENT_SECRET = "DO-NOT-COMMIT"

# Python
import os
domain = os.environ["AUTH0_DOMAIN"]
```

API authorization pattern

```
def require_scope(token_scopes, required_scope):
if required_scope not in token_scopes:
raise PermissionError("403: insufficient scope")
return True
```

JML validation pattern

```
ALLOWED_DEPARTMENTS = {"HR", "Finance", "IT", "Sales"}

def validate_employee(row):
required = {"username", "department", "title"}
missing = required - row.keys()
if missing:
raise ValueError(f"Missing fields: {sorted(missing)}")
if row["department"] not in ALLOWED_DEPARTMENTS:
raise ValueError("Invalid department")
return True
```

Logging pattern

```
import logging

logging.basicConfig(
filename="iam-lab.log",
level=logging.INFO,
format="%(asctime)s %(levelname)s %(message)s"
)

logging.info("JML action started: %s", username)
```

Security rule

Use environment variables or a local secret store for credentials. Add .env, token files, and screenshots containing secrets to .gitignore. Never upload a real bearer token.

8. IAM Testing Matrix

Test ID	Scenario	Expected	Evidence
AUTH-01	Valid user login	Success	Screenshot/log
AUTH-02	Invalid password	Failure	Log
AUTH-03	MFA/passwordless test	Additional verification	Screenshot
API-01	Valid access token	200	Postman/Bruno
API-02	No token	401	API response
API-03	Wrong audience	401	API response
API-04	Missing scope	403	API response
JML-01	Create new user	Created once	Log
JML-02	Duplicate user	Handled safely	Log
JML-03	Mover Finance→IT	Old access removed	Before/after
JML-04	Leaver	Authentication blocked	Test result
AUD-01	Stale account	Finding created	Report
AUD-02	Excess access	Finding + remediation	Before/after
AUD-03	Lifecycle mismatch	Finding created	Report

A hiring manager should be able to reproduce at least the major tests from your README without asking you for undocumented steps.

9. GitHub Portfolio Structure

```
securehealth-auth0-iam/
■■■ README.md
■■■ architecture/
■ ■■■ iam-architecture.png
■ ■■■ oidc-flow.png
■ ■■■ jml-flow.png
■■■ docs/
■ ■■■ IAM-Requirements.md
■ ■■■ Lifecycle-SOP.md
■ ■■■ Permission-Matrix.md
■ ■■■ MFA-Design.md
■ ■■■ SSO-SCIM-Design.md
■ ■■■ Audit-Executive-Summary.md
■■■ scripts/
■ ■■■ provision_user.py
■ ■■■ move_user.py
■ ■■■ offboard_user.py
■ ■■■ generate_access_report.py
■■■ api/
■ ■■■ main.py
■ ■■■ auth.py
■■■ tests/
■ ■■■ auth-tests.md
■ ■■■ api-tests.md
■ ■■■ jml-tests.md
■■■ reports/
■ ■■■ access-review.csv
■ ■■■ access-review.html
■■■ .gitignore
```

README sections

- Business problem
- Architecture
- Technology stack
- Free-plan assumptions/limitations
- Implementation steps
- Security controls
- JML workflow
- Testing results
- Audit findings
- Lessons learned
- How to reproduce

> Do not write 'implemented enterprise RBAC in Auth0' if you only used custom claims or API scopes. Say exactly what you implemented.

10. Resume & Interview Packaging

Recommended project title

Enterprise IAM & API Access Management Platform | Auth0, OAuth 2.0, OIDC, JWT, Python, FastAPI

Resume bullets — use only after you complete the corresponding lab evidence

- Built a hands-on identity and access management platform using Auth0, implementing OIDC authentication and OAuth 2.0-protected APIs with JWT validation.
- Designed least-privilege API authorization using granular scopes and documented positive/negative access-control test cases.
- Simulated Joiner-Mover-Leaver lifecycle processes and automated user provisioning, access changes, and deactivation using Python with structured logging and error handling.
- Implemented and tested authentication security controls including available MFA/passwordless capabilities and documented tenant feature constraints.
- Produced an IAM access-review report with intentional access findings, remediation evidence, and an executive summary.

Interview demo in 7 minutes

1. 30 sec — State the business problem.
2. 60 sec — Show the architecture.
3. 90 sec — Demonstrate login and token flow.
4. 90 sec — Demonstrate API authorization and a 403 denial.
5. 90 sec — Demonstrate mover/leaver automation.
6. 60 sec — Show access-review finding and remediation.

Questions you should be able to answer

- Why Auth0? What does it provide?
- OAuth vs OIDC vs SAML vs SCIM?
- Why are access tokens used for APIs?
- What validates a JWT?
- What is least privilege?
- How would you handle a mover event safely?
- What should happen when offboarding fails halfway?
- How would you audit privileged access?
- What is the difference between 401 and 403?
- What did your lab teach you that is different from production?

11. How This Improves the Supplied Internship Program

The supplied program's strength is that it treats IAM as operational work: build an environment, automate lifecycle tasks, design least privilege, enforce security, integrate SSO/MFA, generate reports, and finish with an access-review capstone. filecite turn0file0 L2-L17

This Auth0 version preserves those principles while adding the application/API perspective required to prove that identity controls actually protect a resource. It also avoids making paid Auth0 Core RBAC a hard dependency because the current Free plan does not list Role Management as included. cite turn0search0

Original emphasis	Auth0 adaptation
AD forest/OU setup	Auth0 tenant + IAM architecture
AD users/groups	Auth0 users + lifecycle metadata
AD RBAC groups	API scopes + claims/authorization layer
GPO hardening	Authentication/API security tests
Entra SAML SSO	OIDC + optional enterprise SSO
Conditional Access MFA	Available Auth0 MFA/passwordless controls + documented limits
PowerShell automation	Python + Auth0/API automation
AD access review	Auth0/API identity and permission review
Bank UAR capstone	SecureHealth quarterly access review

12. Final Completion Checklist

Checkpoint	Done?
Auth0 tenant secured and documented	☐
GitHub repository created	☐
Architecture diagram completed	☐
Synthetic users created	☐
Joiner/Mover/Leaver workflow tested	☐
Custom API and scopes configured	☐
JWT validation implemented	☐
401/403 tests completed	☐
OIDC login working	☐
MFA/passwordless capability tested where available	☐
SSO/SCIM module completed or documented with free alternative	☐
Python JML scripts completed	☐
Audit report generated	☐
Three findings documented	☐
One finding remediated	☐
AD hybrid mapping completed	☐
README reproducible	☐
Resume bullets backed by evidence	☐
7-minute interview demo rehearsed	☐

Completion standard

You are finished when another person can clone the repository, understand the architecture, reproduce the major tests, and you can explain every design decision without reading your notes.

Current Auth0 reference note

Auth0's public pricing currently lists the Free plan at $0/month and includes 25,000 MAUs, 5 Organizations, one Enterprise Connection, inbound SCIM, passwordless authentication, custom attributes, and 5 Actions; Role Management is listed as not included. Re-check the tenant's subscription page before starting modules that depend on a feature because product entitlements can change. □cite□turn0search0□

This curriculum is adapted from the supplied IAM Analyst Internship Program and is for skill-building. The source itself states that it is a self-directed training curriculum and not an official employer document. □filecite□turn0file0□L230-L234□